Test Plan

1. Document Control

- **Project Name:** TfL “What-If” Simulation
- **Version:** 0.3
- **Author (Testing Lead):** Sattyaj Paul
- **Date:** 23/01/2026
- **Status:** Draft

2. Introduction

2.1 Purpose

This document describes the testing strategy, scope, and approach for the project. It defines how testing will be planned, executed, and managed throughout the software development lifecycle.

2.2 Scope

This test plan applies to all features derived from approved user stories. At this early stage, the focus is on validating requirements, planning test coverage, and defining processes rather than detailed test cases.

- Validation for user interface through the graphical user interface
- Verification that map of London is correctly shown to the user
- Visual design, styling, and accessibility compliance are out of scope unless explicitly required.

3. References

- Project Requirements / User Stories
- GitLab Repository
- University / Industry Software Engineering Standards

4. Test Strategy

4.1 Testing Levels

- **Unit Testing:** Developer-written tests for individual functions/ modules.
- **Integration Testing:** Verifying interactions between components.
- **System Testing:** End-to-end testing against user stories.
- **User Acceptance Testing:** Validation against acceptance criteria.
- **User Interface Testing:** Validating correct user interactions, input handling, and display of simulation results.

4.2 Test Types

- Functional Testing
- Regression Testing
- Smoke Testing
- (Optional later) Performance & Security Testing
- Usability-focused UI Functional testing
- Manual Testing

4.3 Test Approach

Testing will be risk-based and aligned with user stories. Each user story will include acceptance criteria that directly map to test cases. For the user interface, testing will focus on critical user flows such as selecting stations, applying disruptions, and viewing simulation results. UI testing will prioritise functional correctness over visual appearance.

4.4 UI Testing Considerations

- UI test will be primarily manual and exploratory
- Automated UI testing may be added if time permits

5. Test Items

The following components will be tested:

- Core application features
- Authentication & authorisation (Only if implemented)
- Data validation and error handling
- User Interface (input handling, result display, error messages)

6. Test Environment

- **Version Control:** GitLab
- **Test Frameworks:** JUnit, PyTest, Jest
- **Frontend:** React, Vite
- **Database:** Neo4j
- **Browser:** Chrome
- **CI Pipeline:** GitLab CI

7. Entry & Exit Criteria

7.1 Entry Criteria

- User stories are approved
- Acceptance criteria defined
- Code committed to repository

7.2 Exit Criteria

- All planned tests executed
- No critical defects open
- Test summary report completed

8. Test Deliverables

- Test Plan
- Test Cases
- Automated Test Scripts
- Test Reports

9. Roles & Responsibilities

- **Testing Lead:** Test planning, coordination, reporting, Unit testing
- **Document Lead:** Structure, Consistency, Version Control, Documentation Quality
- **Project Management Lead:** Planning, coordinating tasks, tracking progress, delivered on time
- **Software Development Lead:** Implementation, code quality,

10. Defect Management

- Defects logged in GitLab Issues
- Each defect includes steps to reproduce, severity, and screenshots/logs

11. Risks & Mitigation

Risk	Impact	Mitigation
Unclear requirements	High	Early review of user stories
Limited Time	Medium	Prioritised Testing